

Project Report

Traffic Light Controller with Pedestrian Request Handling

Overview

This project implements a parameterized Traffic Light Controller (TLC) in Verilog using a finite state machine (FSM) architecture. The controller manages vehicle traffic signals and pedestrian crossing requests while ensuring safe state transitions, deterministic timing behavior, and robust recovery from invalid operating conditions.

Unlike a basic traffic light FSM that services requests only when they occur at specific points in the cycle, this implementation incorporates a request-latching mechanism that stores pedestrian crossing requests until they can be safely serviced. The design also includes fault-state recovery logic and a diagnostic blinking-yellow indication for invalid state detection.

The project consists of a synthesizable RTL implementation and a self-checking verification environment.

Design Objectives

The controller was designed with the following goals:

- Deterministic traffic signal sequencing
- Configurable signal timing parameters
- Reliable pedestrian request servicing
- Protection against lost button presses
- Safe reset behavior
- Fault recovery from illegal FSM states
- Simple verification and observability

System Architecture

Top-Level Module

tlc

The entire controller is implemented as a single RTL module containing:

- Finite State Machine
- Timing Counter
- Pedestrian Request Latch
- Button Edge Detector
- Fault Indicator Generator

A separate testbench instantiates and verifies the controller.

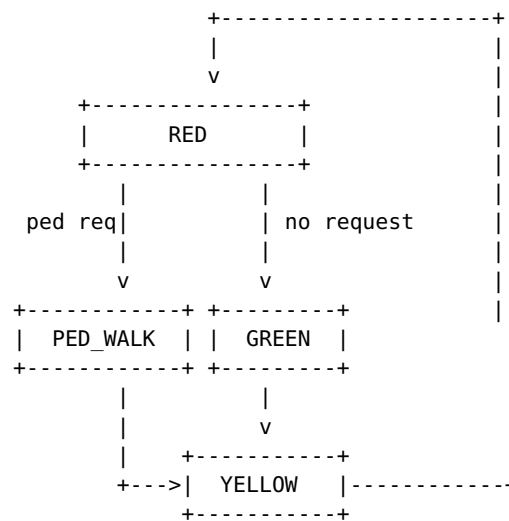
Finite State Machine

State Encoding

State	Encoding	Description
S_RED	2'b00	Vehicles stopped
S_YELLOW	2'b01	Transition warning
S_GREEN	2'b10	Vehicles allowed
S_PED_WALK	2'b11	Pedestrian crossing phase

The controller operates as a Moore FSM where outputs depend solely on the current state.

State Transition Diagram



Timing Architecture

The controller uses a single timing counter that measures the duration spent in each state.

Configurable Parameters

```
parameter T_RED = 100;
parameter T_YELLOW = 15;
parameter T_GREEN = 60;
parameter T_PED_WALK = 30;
```

These parameters define the dwell time for each phase and can be adjusted at synthesis time without modifying the FSM implementation.

Pedestrian Request Handling

Edge Detection

Pedestrian requests are generated only on the rising edge of the push button.

```
ped_btn && !ped_btn_prev
```

This prevents repeated requests while the button remains pressed.

Benefits: Eliminates duplicate requests, avoids repeated triggering from a held button, simplifies request tracking.

Request Latching

A dedicated request register stores pending crossing requests.

```
reg ped_req;
```

Whenever a valid button edge is detected, the request is latched and remains active until the controller begins servicing the pedestrian crossing phase.

Operational Behavior

If a request arrives during GREEN, YELLOW, or RED, the request is preserved and serviced later when the controller reaches the decision point at the end of the RED phase.

This mechanism prevents missed requests and decouples pedestrian input timing from the FSM transition timing.

Transition-Boundary Protection

An additional design detail is the handling of requests that arrive exactly when the RED phase expires.

At the RED-state decision point, the controller evaluates:

```
ped_req || (ped_btn && !ped_btn_prev)
```

This allows a newly arriving button press to be serviced immediately even if it occurs during the same cycle as the RED-state timeout.

Without this logic, requests occurring at the state-transition boundary could be lost.

State Behavior

RED

Outputs: red=1, yellow=0, green=0, ped_cross=0

Transition Conditions: After the RED timer expires: if a pedestrian request exists → PED_WALK, otherwise → GREEN

GREEN

Outputs: red=0, yellow=0, green=1, ped_cross=0

Transition Conditions: After T_GREEN cycles: GREEN → YELLOW

YELLOW

Outputs: red=0, yellow=1, green=0, ped_cross=0

Transition Conditions: After T_YELLOW cycles: YELLOW → RED

PED_WALK

Outputs: red=1, yellow=0, green=0, ped_cross=1

Vehicle traffic is halted while pedestrians are allowed to cross.

Transition Conditions: After T_PED_WALK cycles: PED_WALK → GREEN

Counter Architecture

The controller uses a single shared counter.

```
reg [7:0] counter;
```

The counter increments once per clock cycle, tracks residency time in the current state, and resets whenever a state transition occurs.

Using a single counter reduces register count and simplifies timing management compared to maintaining independent timers for each state.

Fault Recovery Mechanism

Invalid State Detection

The FSM includes explicit handling for illegal state values through its default branch.

When an invalid state is encountered:

1. Normal traffic outputs are disabled.
2. Yellow enters a blinking diagnostic mode.
3. Internal state is forced back to S_RED.
4. The timing counter is reset.

This guarantees recovery without requiring an external reset.

Diagnostic Yellow Blink

A dedicated blinking generator produces the fault indication.

Internal Components: blink_ctr, blink_pulse

The blink generator runs independently of the traffic-control FSM and periodically toggles the yellow signal during fault conditions.

Purpose: The blinking yellow output serves as a visible indication that the controller detected an invalid internal state before automatically recovering.

Reset Behavior

The design uses an asynchronous active-low reset.

```
rst_n
```

Upon reset assertion: State = RED, Counter = 0, ped_req = 0

Resulting outputs: red=1, yellow=0, green=0, ped_cross=0

This places the controller in a safe operating configuration immediately, independent of the clock.

Verification Methodology

Testbench Strategy

The verification environment uses a self-checking testbench that automatically validates expected behavior.

To accelerate simulation, reduced timing parameters are used:

```
T_RED = 10  
T_YELLOW = 2  
T_GREEN = 6  
T_PED_WALK = 4
```

Output Validation

A reusable checking task compares DUT outputs against expected values and reports pass/fail status.

Verification messages are generated automatically for each scenario.

Verified Scenarios

Reset Initialization: Verifies that reset forces the controller into the safe RED state.

Pedestrian Request Servicing: Confirms that button presses are correctly stored and serviced.

Transition-Boundary Requests: Validates that requests arriving during the RED-state timeout boundary are not lost.

Pedestrian Walk Completion: Verifies correct return to normal traffic operation after the pedestrian phase completes.

Rapid Button Activity: Exercises the request-handling mechanism under repeated button presses.

Mid-Cycle Reset: Applies asynchronous reset while the FSM is active and confirms immediate recovery.

Fault-State Recovery: Directly injects an invalid FSM state and verifies diagnostic indication, automatic recovery, and return to safe operation. This test specifically validates robustness beyond normal operating conditions.

Resource Characteristics

The design is intentionally lightweight.

Major state elements include:

- FSM state register
- Timing counter

- Pedestrian request latch
- Button history register
- Blink generator registers

No RAM blocks, FIFOs, DSP resources, or complex datapath structures are required.

The implementation is therefore suitable for small FPGA devices and introductory embedded control applications.

Physical Implementation Results

The design was synthesized and implemented on a Xilinx Kintex-7 device (xc7k70t) to validate timing closure and resource footprint on real FPGA fabric.

Metric	Value
Target Device	Kintex-7 (xc7k70t)
Max Frequency	357 MHz
Worst Negative Slack (WNS)	+7.20 ns
LUT / FF Utilization	<1%
Dynamic Power	3 mW

Consistent with the design’s small register count and simple control logic, the controller closes timing with wide positive slack far beyond any practical traffic-signal clock requirement, while consuming negligible FPGA resources and power.

Notable Design Decisions

Queued Pedestrian Requests

Requests are stored until serviced rather than requiring precise timing relative to FSM transitions.

Advantage: Improves usability and prevents missed crossings.

Rising-Edge Request Detection

Button presses generate only a single request regardless of press duration.

Advantage: Prevents repeated servicing from a held button.

Transition-Boundary Handling

New requests are considered during RED-state timeout evaluation.

Advantage: Avoids edge-case request loss.

Automatic Fault Recovery

Illegal states trigger diagnostic behavior and recovery without external intervention.

Advantage: Improves robustness and simplifies debugging.

Possible Future Extensions

The following enhancements are not implemented in the current RTL but could be added in future revisions:

- Separate vehicle and pedestrian signal sets
- Multiple traffic directions
- Sensor-driven adaptive timing
- Emergency vehicle override support
- Programmable timing through memory-mapped registers
- Formal verification using SystemVerilog assertions
- Multi-intersection coordination

Conclusion

This project implements a compact yet robust traffic light controller featuring configurable timing, queued pedestrian request handling, fault recovery logic, and self-checking verification infrastructure. The design demonstrates practical FSM engineering techniques beyond a minimal traffic-light example, particularly through its request-latching mechanism, transition-boundary handling, and diagnostic recovery behavior. Physical implementation on Kintex-7 confirmed timing closure at 357 MHz with negligible resource utilization and power draw, reinforcing its suitability for lightweight FPGA deployment.